

Lesson 2: NumPy Advanced Operations

Các phép toán nâng cao với NumPy

Lesson Information | Thông tin bài học

Item	Description
Duration	90 minutes
Type	Lecture + Lab
Prerequisites	Lesson 1: NumPy Fundamentals

Learning Objectives | Mục tiêu bài học

#	Objective	Assessment
1	Perform element-wise arithmetic operations	Code exercises
2	Understand and apply broadcasting	Quiz
3	Use aggregation and statistical functions	Lab work
4	Apply linear algebra operations	Code exercises

1. Element-wise Operations (20 min)

1.1 Arithmetic Operations

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

# Basic arithmetic (element-wise)
print(a + b)      # [ 6  8 10 12]
print(a - b)      # [-4 -4 -4 -4]
print(a * b)      # [ 5 12 21 32]
print(a / b)      # [0.2  0.33 0.43 0.5 ]
print(a ** 2)     # [ 1  4  9 16]
print(a % 2)      # [1 0 1 0]
print(a // 2)     # [0 1 1 2]

# NumPy functions (equivalent)
print(np.add(a, b))
print(np.subtract(a, b))
print(np.multiply(a, b))
```

```
print(np.divide(a, b))
print(np.power(a, 2))
```

1.2 Comparison Operations

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([5, 4, 3, 2, 1])

print(a == b)      # [False False  True False False]
print(a != b)      # [ True  True False  True  True]
print(a > b)       # [False False False  True  True]
print(a >= b)      # [False False  True  True  True]
print(a < b)       # [ True  True False False False]
print(a <= b)      # [ True  True  True False False]

# Array comparison functions
print(np.array_equal(a, b)) # False
print(np.allclose(a, b))   # False (for float comparison)
```

1.3 Universal Functions (ufuncs)

```
arr = np.array([1, 4, 9, 16, 25])

# Math functions
print(np.sqrt(arr))      # [1.  2.  3.  4.  5.]
print(np.exp(arr))       # [e^1 e^4 ...]
print(np.log(arr))       # Natural log
print(np.log10(arr))     # Log base 10
print(np.log2(arr))      # Log base 2

# Trigonometric
angles = np.array([0, np.pi/6, np.pi/4, np.pi/3, np.pi/2])
print(np.sin(angles))
print(np.cos(angles))
print(np.tan(angles))

# Rounding
arr = np.array([1.2, 2.5, 3.7, 4.1])
print(np.round(arr))     # [1.  2.  4.  4.]
print(np.floor(arr))     # [1.  2.  3.  4.]
print(np.ceil(arr))      # [2.  3.  4.  5.]
print(np.trunc(arr))     # [1.  2.  3.  4.]

# Absolute value
arr = np.array([-1, -2, 3, -4])
print(np.abs(arr))       # [1  2  3  4]
print(np.absolute(arr))  # Same
```

2. Broadcasting (20 min)

2.1 What is Broadcasting?

Broadcasting allows NumPy to perform operations on arrays of different shapes.

Rules:

1. Compare shapes from right to left
2. Dimensions are compatible if they are equal OR one of them is 1
3. Smaller array is "broadcast" across larger array

2.2 Broadcasting Examples

```
# Scalar broadcast
arr = np.array([1, 2, 3, 4])
print(arr + 10)      # [11 12 13 14]
print(arr * 2)       # [2 4 6 8]

# 1D + 1D (same shape)
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)         # [11 22 33]

# 2D + 1D
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6]])
row = np.array([10, 20, 30])
print(arr2d + row)
# [[11 22 33]
#  [14 25 36]]
```

2.3 Broadcasting Visualization

Array A (3, 4): <pre>[[1 2 3 4] [5 6 7 8] [9 10 11 12]]</pre> + Result (3, 4): <pre>[[11 22 33 44] [15 26 37 48] [19 30 41 52]]</pre>	Array B (4,): <pre>[10 20 30 40]</pre> ↓ broadcast <pre>[[10 20 30 40] [10 20 30 40] [10 20 30 40]]</pre> =
--	--

2.4 Column Broadcasting

```

arr2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

# Add column vector
col = np.array([[10],
                [20],
                [30]])

# Or: col = np.array([10, 20, 30]).reshape(-1, 1)

print(arr2d + col)
# [[11 12 13]
#  [24 25 26]
#  [37 38 39]]

```

2.5 Broadcasting Errors

```

a = np.array([[1, 2, 3],
              [4, 5, 6]]) # Shape: (2, 3)

b = np.array([1, 2])      # Shape: (2,)

# This will ERROR – shapes (2,3) and (2,) not compatible
# print(a + b) # ValueError

# Fix: reshape b
b = b.reshape(-1, 1)      # Shape: (2, 1)
print(a + b)              # Now works!

```

3. Aggregation Functions (20 min)

3.1 Basic Aggregations

```

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

# Sum
print(np.sum(arr))          # 78 (all elements)
print(np.sum(arr, axis=0))  # [15 18 21 24] (sum of columns)
print(np.sum(arr, axis=1))  # [10 26 42] (sum of rows)

# Product
print(np.prod(arr))         # Product of all elements

# Cumulative

```

```
print(np.cumsum(arr))      # Cumulative sum
print(np.cumprod(arr))     # Cumulative product
```

3.2 Statistical Functions

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Central tendency
print(np.mean(arr))        # 5.5
print(np.median(arr))      # 5.5
# Mode: use scipy.stats.mode()

# Spread
print(np.std(arr))         # Standard deviation
print(np.var(arr))         # Variance
print(np.ptp(arr))         # Peak-to-peak (max - min)

# Min/Max
print(np.min(arr))         # 1
print(np.max(arr))         # 10
print(np.argmin(arr))      # 0 (index of min)
print(np.argmax(arr))      # 9 (index of max)

# Percentile/Quantile
print(np.percentile(arr, 25)) # 3.25 (25th percentile)
print(np.percentile(arr, 50)) # 5.5 (median)
print(np.percentile(arr, 75)) # 7.75
print(np.quantile(arr, 0.25)) # Same as percentile(arr, 25)
```

3.3 Aggregation with Axis

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# axis=0: operate along rows (result is per column)
print(np.mean(arr, axis=0)) # [4. 5. 6.]

# axis=1: operate along columns (result is per row)
print(np.mean(arr, axis=1)) # [2. 5. 8.]

# keepdims - maintain dimensions
print(np.mean(arr, axis=1, keepdims=True))
# [[2.]
#  [5.]
#  [8.]]
```

3.4 Boolean Aggregations

```

arr = np.array([1, 2, 3, 4, 5])

# Check conditions
print(np.any(arr > 3))      # True (at least one)
print(np.all(arr > 3))     # False (not all)
print(np.any(arr > 10))    # False

# Count
print(np.sum(arr > 3))     # 2 (count True values)
print(np.count_nonzero(arr > 3)) # 2

# Where (find indices)
print(np.where(arr > 3))   # (array([3, 4]),)
print(np.where(arr > 3, 'big', 'small'))
# ['small' 'small' 'small' 'big' 'big']

```

4. Linear Algebra (20 min)

4.1 Matrix Operations

```

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

# Matrix multiplication
print(np.dot(A, B))
# [[19 22]
#  [43 50]]

# Or use @ operator (Python 3.5+)
print(A @ B)

# Element-wise multiplication (not matrix mult!)
print(A * B)
# [[ 5 12]
#  [21 32]]

# Transpose
print(A.T)
# [[1 3]
#  [2 4]]

# Trace (sum of diagonal)
print(np.trace(A)) # 5

# Determinant
print(np.linalg.det(A)) # -2.0

```

```
# Inverse
print(np.linalg.inv(A))
# [[-2.   1. ]
#   [ 1.5 -0.5]]

# Verify: A @ A_inv = I
print(A @ np.linalg.inv(A))
```

4.2 Solving Linear Equations

```
# Solve: Ax = b
# 2x + 3y = 8
# 3x + 4y = 11

A = np.array([[2, 3],
              [3, 4]])
b = np.array([8, 11])

x = np.linalg.solve(A, b)
print(x) # [1. 2.] -> x=1, y=2

# Verify
print(A @ x) # [8. 11.]
```

4.3 Eigenvalues and Eigenvectors

```
A = np.array([[4, 2],
              [1, 3]])

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)

# Verify: A @ v = λ @ v
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]
    λ = eigenvalues[i]
    print(f"A @ v: {A @ v}")
    print(f"λ * v: {λ * v}")
```

4.4 Matrix Decomposition

```
# SVD - Singular Value Decomposition
A = np.array([[1, 2, 3],
```

```

[4, 5, 6]])

U, s, Vt = np.linalg.svd(A)
print("U:", U.shape)    # (2, 2)
print("s:", s.shape)    # (2,) singular values
print("Vt:", Vt.shape)  # (3, 3)

# Reconstruct A
S = np.zeros((2, 3))
S[:2, :2] = np.diag(s)
A_reconstructed = U @ S @ Vt

# QR Decomposition
Q, R = np.linalg.qr(A.T)

# Cholesky Decomposition (for positive definite matrices)
# A = L @ L.T

```

4.5 Norms

```

v = np.array([3, 4])
A = np.array([[1, 2], [3, 4]])

# Vector norms
print(np.linalg.norm(v))      # L2 (Euclidean): 5.0
print(np.linalg.norm(v, 1))   # L1 (Manhattan): 7
print(np.linalg.norm(v, np.inf)) # L∞ (Max): 4

# Matrix norms
print(np.linalg.norm(A))      # Frobenius norm
print(np.linalg.norm(A, 'fro')) # Same

```

5. Advanced Techniques (10 min)

5.1 Sorting

```

arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])

# Sort
print(np.sort(arr))          # [1 1 2 3 4 5 6 9]
print(np.argsort(arr))       # [1 3 6 0 2 4 7 5] (indices)

# Sort 2D
arr2d = np.array([[3, 1, 2],
                  [6, 4, 5]])
print(np.sort(arr2d, axis=0)) # Sort each column
print(np.sort(arr2d, axis=1)) # Sort each row

```



```
# Partial sort
print(np.partition(arr, 3))    # First 3 smallest, rest unsorted
```

5.2 Set Operations

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([3, 4, 5, 6, 7])

print(np.intersect1d(a, b))    # [3 4 5]
print(np.union1d(a, b))       # [1 2 3 4 5 6 7]
print(np.setdiff1d(a, b))     # [1 2]
print(np.setxor1d(a, b))      # [1 2 6 7]
print(np.in1d(a, b))          # [False False True True True]
```

5.3 Copying vs Views

```
arr = np.array([1, 2, 3, 4, 5])

# View (shares memory)
view = arr[1:4]
view[0] = 100
print(arr)    # [1 100 3 4 5] - original changed!

# Copy (independent)
arr = np.array([1, 2, 3, 4, 5])
copy = arr[1:4].copy()
copy[0] = 100
print(arr)    # [1 2 3 4 5] - original unchanged
```

Practice Exercises | Bài tập thực hành

Exercise 2.1: Broadcasting Practice

```
# Create a 5x5 distance matrix where element [i,j]
# is the absolute difference |i - j|
# Expected:
# [[0 1 2 3 4]
#  [1 0 1 2 3]
#  [2 1 0 1 2]
#  [3 2 1 0 1]
#  [4 3 2 1 0]]
```

Exercise 2.2: Statistics

```
# Given: 100 random exam scores (0-100)
scores = np.random.randint(0, 101, 100)

# Calculate:
# 1. Mean, median, std
# 2. Number of students who passed (≥50)
# 3. Percentage of A grades (≥90)
# 4. Grade distribution (A, B, C, D, F)
```

Exercise 2.3: Linear Algebra

```
# 1. Solve the system:
#   x + 2y + 3z = 14
#   2x + 5y + 3z = 18
#   4x + 6y + 8z = 32

# 2. Find eigenvalues of:
#   [[4, -2],
#    [1,  1]]
```

Exercise 2.4: Image Processing Simulation

```
# Create a 100x100 "image" (random values 0-255)
# 1. Normalize to 0-1 range
# 2. Apply threshold (>0.5 = 1, else 0)
# 3. Count white and black pixels
# 4. Calculate mean intensity per row
```

Summary | Tóm tắt

Key Operations

Category	Functions
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code> , <code>@</code>
Comparison	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>
Aggregation	<code>sum</code> , <code>mean</code> , <code>std</code> , <code>min</code> , <code>max</code>
Linear Algebra	<code>dot</code> , <code>inv</code> , <code>solve</code> , <code>eig</code>

Broadcasting Rules

- 1. Compare shapes right-to-left
- 2. Dimensions must be equal or one must be 1

3. Smaller array broadcasts to match larger

Next Lesson | Bài tiếp theo

Lesson 3: Pandas Basics

- Series and DataFrame
- Reading data files
- Data selection and filtering